

What is a training example?

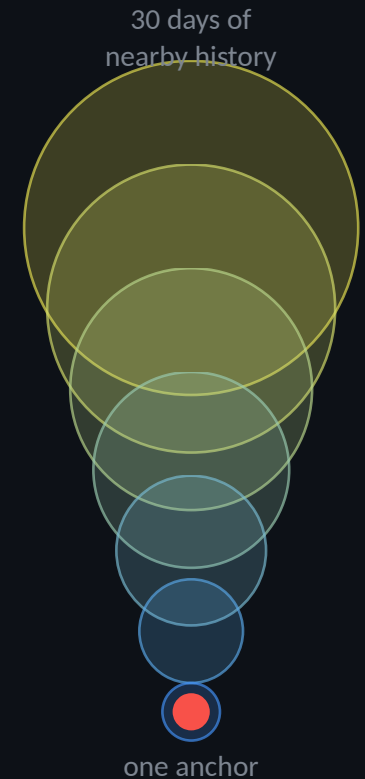
The cone codec's example, its boundaries, and where it should be materialized.

E-069 — the cone-native codec: an encoder that reads 30 days of nearby history around one ocean cell, so that motion can live inside the embedding.

2026-09-03

Proposal in one line

Keep generating examples on the fly — but make every example addressable, dumpable and certifiable.
Materialize the keys and the evaluation set, not the bytes.



One anchor, plus everything around it

A training example is **one anchor** — one ocean cell and one 5-day bin — together with the cone of history that surrounds it.

42

patch tokens

the 3×3 neighbourhood at lag 0, one per channel

706

dot tokens

single values at (channel, lag 1–6, offset) inside the inner cone

42 + 84

targets

the anchor's own values now, and at t+1 and t+2

4

context numbers

season (sin/cos of day-of-year), latitude, longitude

6,579 numbers ≈ 19 KB

Nothing is stored per example: it is gathered by index from the 36 GB tensor at batch time.

750 tokens enter attention (two bookkeeping tokens + 42 + 706) · a batch of 256 examples ≈ 6 MB

Seven bins of history, two bins of future



reach grows with lag — $129.6 \times (1 + \text{lag})$ km for currents and sea-surface height

A wind stress, storminess

8 dots · 500 km at lags 0–1, then nothing

B currents, sea-surface height, Argo column

80 dots · disc grows $129.6 \times (1 + \text{lag})$ km

C sea-surface temperature, mixed layer

81 dots · 500 → 388.8 → ... → 907 km

rg Argo depth channels

6 dots · anchor column only, no disc

Dependency cone (E-069)

388.8 km
family B · inner cone

123
dots per channel

1036
tokens per anchor · 42
patch + 994 dots

what-if geometry — not what the codec was trained on

family B · currents & SSH

lag sweep lags

lag 2 · 10 days back · 2017-05-27 held-out year
anchor 36.00° N 70.00° W · 15 dots drawn

date 06/06/2017

☐ follow the cone — drive the globe's date

☒ live on the app's Cones tab

The dataset is a distribution, not a list of files

thing	size	when made	keyed by
the input tensor (family 4 r3)	35.7 GB float16	once, sha-pinned (fa460837fa)	its sha256
the anomaly-space scratch copy	36 GB	per run, ~40 min	(per run)
per-row dot tables (281 rows)	tiny	per run, 0.8 s	geometry constants
the observed mask	computed per gather	—	—
the fixed evaluation anchor set	1,024 anchors	per run, from seed+991	seed
the velocity-probe anchors	2,048	per run, seed+7717	seed
the training anchor stream and mask draws	implied	per run, from the seed	seed
(stage 2) the embedding cache Z	17.4 GB float16	once per codec × tensor	codec-weight hash × tensor sha

Everything else — the example itself — is derived on the fly.

Uniform draws from 251 million anchors

2,891

of 3,142 bins are admissible

all nine bins $t-6 \dots t+2$ must exist and be training bins

$\approx 86,700$

ocean cells

any cell where a current was ever observed

$\approx 251 \text{ M}$

anchors in the pool

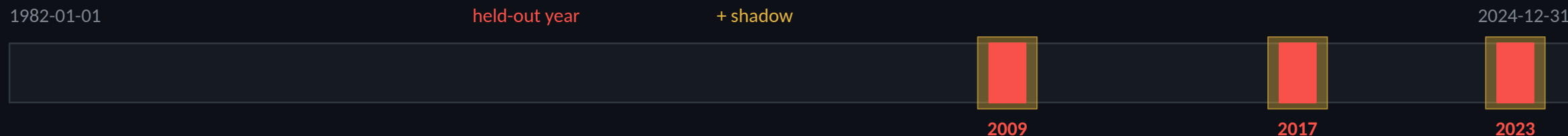
drawn uniformly, with replacement — there are no epochs

What a $20,000 \times 256$ run actually sees

5.1 million anchors = 2 % of the pool

- 251 bins excluded: the three held-out years 2009 / 2017 / 2023 with their 6-bin shadows, and the axis ends.
- No duplication in storage, and near-none in sampling — but every tensor value is read by roughly 700 different examples.
- That 700 $\times$ is exactly the duplication a materialized dataset would have to pay in bytes.

The held-out years, and the shadow around them



Zoom — one year edge, in 5-day bins



Training — certified

The whole cone ($t-6 \dots t+2$) must avoid held-out bins. Enforced twice: by the admissibility rule, and by an independent brute-force certificate over 4,096 drawn anchors before the model is built — 0 violations, or the run refuses to start. Runs #536 and #537 both printed 0.

Evaluation — not certified

Only the anchor bin must be held out; the cone's context may reach into training bins. About 11 % of evaluation anchors sit within 6 bins of a year edge and straddle. Deliberate — “a held-out measurement, not a second training pool” — but an asymmetry.

The axis ends: cones that would run off 1982-01-01 or past 2024-12-31 are refused from every pool, never padded.

The edge of the map, land, and the dateline

0-70°N



○ off-window dot — kept as a token, valid = false, value 0, and never a target

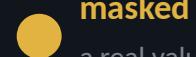
The global family (E-070) will wrap at the dateline — a 10-line change, planned.

Three kinds of token



real

a measured value the model may read



masked

a real value we hid on purpose — and ask for back



missing

land, never observed, or off the window — never a target

Existence is enforced in exactly one place: one key-padding mask. A test flips an invalid dot's value and asserts the embedding is bit-identical.

Observation gaps are represented, never imputed

≈ 83 %

of depth-channel dots carry the missing token — even inside the Argo era

Argo is live one bin per month — one pentad in six



five months of 5-day bins — filled bins are the ones Argo actually reports

22 of 43 years carry no live Argo bins at all.

- Missing is never forward-filled: telling the model the subsurface was observed on a day it was not would be a lie in the data.
- An unobserved target has weight exactly 0 — the decoder is never scored against a made-up zero.
- The token shape stays fixed (706 dots at every latitude); liveness rides in a flag, not in the shape.

Missingness is information — so it is a token, not a hole.

What the example is actually used for

Hide

- **whole channels** — currents 50 %, others 30 %
- **a band of recent lags** — 30 % of examples, lags 1 ... l_0
- **a 90° sector** — 30 % of examples

Score

Gaussian negative log-likelihood, weighted by family (A 1.0, B 2.0, C 1.5). The headline term reads the 32-number embedding z alone.

Ask 382 questions

- 42** reconstruct the anchor's own values
- 84** predict the future values at $t+1$ and $t+2$
- 256** recover hidden dots from elsewhere in the cone

The snapshot twin ($L_{in} = 0$)

44 tokens, no history, still asked for $t+1$ and $t+2$ — the H1 control: does the embedding carry velocity?

The forecaster's example is a different object

A sequence of embeddings, not raw values

- Reads 144 lags (2 years) of embeddings at the anchor column, plus the outer spiral — 24 cells per lag from lag 7.
- Embeddings come from the materialized Z cache (17.4 GB), gathered on the fly per window.
- Off-window cells are zero-filled; windows must not touch any held-out bin, including each frame's teacher-forced target.
- That rule was derived once and then re-counted by brute force: 2,779 → 2,417 usable end-bins.

3,432

cells read per window — the new annulus stencil

20,880

cells read by the older full cylinder

Status: the annulus stencil is the one E-069 piece not yet written.

Generate on the fly, or materialize every example?

Option 1 — generate on the fly (today)

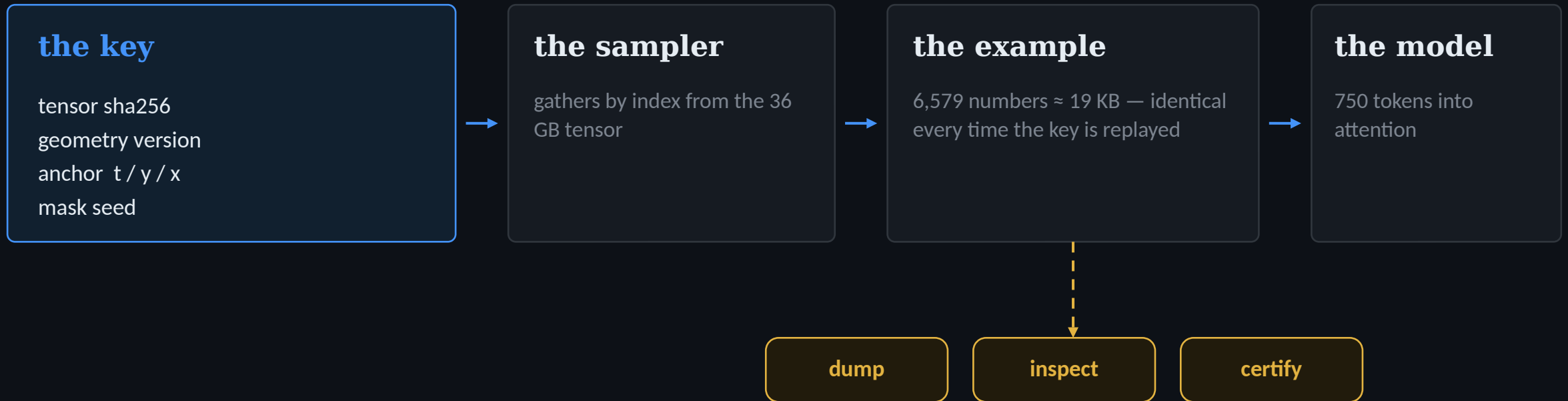
Option 2 — materialize each example

Storage	0 per example	251 M × 19 KB ≈ 4.8 TB for the pool; one run’s stream ≈ 97 GB
Duplication	none	700× — each cell sits inside ~700 cones
What an example is	a pure function of (tensor sha, geometry, anchor, mask seed)	bytes on disk, with boundary flags baked in
Boundary handling	the sampler + a certificate that runs every dispatch	the pipeline — but a sampler bug becomes permanent data
Changing the geometry	costs nothing	rebuilds everything
Inspectability	not inspectable without running code	examples are files you can open; sequential reads are fast

Runtime cost of option 1: 0.2–0.8 ms per anchor of host gather — about 1 hour of a 20,000-step run, not overlapped with the GPU today.

Materialize the key, not the bytes

An example is fully determined by a **small key**. Materializing the bytes is caching; materializing the key gives the same inspectability at zero storage — provided a tool turns a key back into the example, bit-identically.



The repo already works this way: the stage-2 cache is keyed by two hashes, and the JAX port is certified by gates rather than by re-reading weights.

What I would do — about one day of work

1 A dump-by-key tool

Dump any example by key — or by “run #N, step s, batch index b”, replaying the seeded stream — to NPZ, JSON and a picture. The Cones tab gains a “show a real example” mode.

2 A run manifest

Tensor sha, geometry json sha, seed and step count written into every run’s record, so any example ever trained on is regenerable.

3 Materialize the evaluation set

The 1,024 evaluation anchors and their fixed mask draws, as a versioned file shipped with each run. This is where materialization pays.

4 An executable boundary spec

Property tests sweeping every edge row and column, both axis ends and every holdout edge; the certificate extended to evaluation anchors behind `--eval-scope {anchor, window}`.

5 Materialize the observed mask

Once per tensor — a 2.2 GB bitmask — and precompute the row tables. Removes the double read and part of the 0.2–0.8 ms.

6 An escape hatch

`--materialize-stream` writes a run’s exact examples (≈ 97 GB) for audits. Never for training.

Cost: about one day of implementation • no change to E-069’s numbers • the evaluation file becomes an artefact.

Boundary handling under each option

BOUNDARY	ENFORCED TODAY	UNDER OPTION 2	HOW IT IS PROVEN
time / held-out years	the admissibility rule in the sampler	a filter at build time	certificate over 4,096 anchors, before every run
window edge	dot kept, valid = false; one key-padding mask	a flag baked into each stored example	test: flip an invalid dot, embedding is bit-identical
land / never observed	the missing token, at gather time	a missing flag written once at build	by construction, plus gather tests
dateline / wrap	no wrap; E-070 will wrap — a 10-line change	a full rebuild of every byte	geometry test (planned)
axis ends 1982 / 2024	cones refused from every pool, never padded	rows simply absent from the built set	admissibility count + the same certificate

Under option 1 the proof is a test that runs every dispatch. Under option 2 the proof is a build that ran once.

Four questions for you

Generation

Keep on-the-fly generation, as proposed — or materialize a run's stream by default?

Evaluation scope

Keep “anchor bin held out, context may straddle” — or measure the strict variant too?

A public gallery

A small gallery of real examples — say 50, about 1 MB — for the Cones tab and the paper. Yes or no?

Order of work

After run #537's H1 reading, or in parallel? It touches no training code path except the evaluation-set file.

Recommendation: parallel, small, and the strict evaluation variant measured once — so the straddle's effect is a number rather than an assumption.

What is a training example?

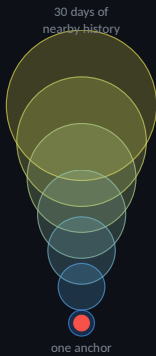
The cone codec's example, its boundaries, and where it should be materialized.

E-069 — the cone-native codec: an encoder that reads 30 days of nearby history around one ocean cell, so that motion can live inside the embedding.

2026-09-03

Proposal in one line

Keep generating examples on the fly — but make every example addressable, dumpable and certifiable.
Materialize the keys and the evaluation set, not the bytes.



This deck exists because of one question: what exactly is a training example here, and where should it live? E-069 is the internal name of the cone-native codec, which is an encoder that reads about thirty days of nearby history around a single ocean cell so that motion can live inside the embedding it produces. The question has two extreme answers, and everything in between is a spectrum: at one end we generate every example on the fly, as we do today; at the other end we write every example out to disk before training starts. This deck first answers what an example actually is, in numbers, then walks its three boundaries, then weighs the two options honestly, and finally makes a proposal. The proposal in one line is this: keep generating examples on the fly, but make every example addressable, dumpable and certifiable. In other words, materialize the small key that identifies an example, and materialize the evaluation set, but do not materialize the bytes of every example. Every number in this deck was read out of the code rather than remembered, and the reading was done on the third of September 2026.

One anchor, plus everything around it

A training example is **one anchor** — one ocean cell and one 5-day bin — together with the cone of history that surrounds it.

42

patch tokens

the 3x3 neighbourhood at lag 0, one per channel

706

dot tokens

single values at (channel, lag 1-6, offset) inside the inner cone

42 + 84

targets

the anchor's own values now, and at t+1 and t+2

4

context numbers

season (sin/cos of day-of-year), latitude, longitude

6,579 numbers ≈ 19 KB

Nothing is stored per example: it is gathered by index from the 36 GB tensor at batch time.

750 tokens enter attention (two bookkeeping tokens + 42 + 706) · a batch of 256 examples = 6 MB

What is a training example?

2

A training example is one anchor plus its surroundings, and an anchor is one ocean grid cell at one five-day time bin. A five-day bin is called a pentad, and the whole record is chopped into pentads rather than days. A channel is one of the forty-two measured quantities we carry for every cell: the currents, sea-surface height, the mixed-layer depth, thirty-two Argo temperature and salinity levels down the water column, wind stress, storminess, sea-surface temperature, and the two new current components. A lag is simply how many bins back in time a value comes from, so lag one is five days before the anchor and lag six is thirty days before it. Around the anchor the example carries forty-two patch tokens, which are the three-by-three neighbourhood at lag zero, one token per channel, and seven hundred and six dot tokens, which are single values picked at a given channel, lag and spatial offset. It also carries what the model must predict: forty-two targets at the anchor itself and eighty-four future targets, being the anchor's own values one and two bins ahead. Four context numbers finish it off: the season written as the sine and cosine of the day of year, plus latitude and longitude. Altogether that is six thousand five hundred and seventy-nine numbers, about nineteen kilobytes, and none of it sits on disk — it is gathered by index out of the thirty-six gigabyte tensor at the moment the batch is built.

PICTURE OF ONE EXAMPLE

Seven bins of history, two bins of future



This is one example drawn to scale in time. The horizontal line is time in five-day bins, running from six bins before the anchor on the left to two bins after it on the right; the anchor bin, marked in red, is the one labelled t. At each bin behind the anchor we draw a disc, and the disc is the region the model is allowed to look at for that lag; the further back in time, the wider the disc, because a driver moving at about a third of a metre per second can travel that much further before it reaches the anchor. For the currents and for sea-surface height, which is how high the sea stands, the radius is one hundred and twenty-nine point six kilometres multiplied by one plus the lag, so it reaches nine hundred and seven kilometres at the oldest bin. Wind is different: it is read out to five hundred kilometres at lags zero and one and then stops, because the wind’s useful memory in this system is only about ten days. Sea-surface temperature and the mixed-layer depth start at five hundred kilometres, drop to three hundred and eighty-nine at lag two when the atmosphere’s fast stirring drops out, and then widen again to nine hundred and seven as the slower ocean memory takes over. The Argo depth channels, which are temperature and salinity measured down the water column by drifting floats, get only the anchor’s own column and no disc at all, because Argo reports roughly one bin in six and a full disc would be about ninety per cent empty. At the anchor itself we add a three-by-three patch of neighbouring cells, one token per channel, and to the right sit the two future bins the model is asked to predict.

The dataset is a distribution, not a list of files

thing	size	when made	keyed by
the input tensor (family 4 r3)	35.7 GB float16	once, sha-pinned (fa460837fa)	its sha256
the anomaly-space scratch copy	36 GB	per run, ~40 min	(per run)
per-row dot tables (281 rows)	tiny	per run, 0.8 s	geometry constants
the observed mask	computed per gather	—	—
the fixed evaluation anchor set	1,024 anchors	per run, from seed+991	seed
the velocity-probe anchors	2,048	per run, seed+7717	seed
the training anchor stream and mask draws	implied	per run, from the seed	seed
(stage 2) the embedding cache Z	17.4 GB float16	once per codec × tensor	codec-weight hash × tensor sha

Everything else — the example itself — is derived on the fly.

This table is the honest inventory of what actually sits on disk today. The big one is the input tensor, thirty-five point seven gigabytes of half-precision numbers, built once and pinned by its sha256 — a sha256 is a fingerprint of the exact bytes, so if a single number changed the fingerprint would change too. Each run makes a thirty-six gigabyte scratch copy in anomaly space, meaning the seasonal cycle has been subtracted, and that takes about forty minutes. The little per-row dot tables, two hundred and eighty-one rows of them, are geometry lookups rebuilt in under a second. The evaluation anchors, one thousand and twenty-four of them, and the velocity-probe anchors, two thousand and forty-eight, are not stored either: they are regenerated from the run's random seed, offset by nine hundred and ninety-one and by seven thousand seven hundred and seventeen respectively. The only other large materialized object belongs to stage two: a seventeen point four gigabyte cache of embeddings, keyed by two hashes at once, the codec weights and the tensor. Those two hashes exist because of a real lesson from runs number ten and eleven, and from two machines that each built a tensor with the same name but different bytes — without the double key you can silently train on the wrong embeddings. The point of the table is that the dataset is a distribution you sample from, not a list of files you read.

Uniform draws from 251 million anchors

2,891

of 3,142 bins are admissible

all nine bins $t-6 \dots t+2$ must exist and be training bins

$\approx 86,700$

ocean cells

any cell where a current was ever observed

$\approx 251 \text{ M}$

anchors in the pool

drawn uniformly, with replacement — there are no epochs

What a 20,000 × 256 run actually sees

5.1 million anchors = 2 % of the pool

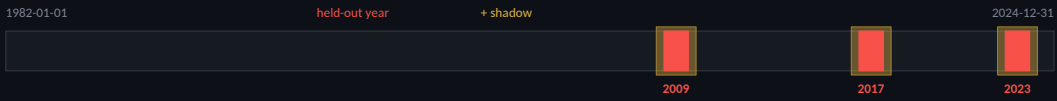
- 251 bins excluded: the three held-out years 2009 / 2017 / 2023 with their 6-bin shadows, and the axis ends.
- No duplication in storage, and near-none in sampling — but every tensor value is read by roughly 700 different examples.
- That 700× is exactly the duplication a materialized dataset would have to pay in bytes.

What is a training example?

5

Anchors are drawn at random, and two admissibility rules decide which ones are allowed. The time rule says every one of the nine bins in the cone, from six before the anchor to two after it, must exist and must be a training bin; that leaves two thousand eight hundred and ninety-one admissible bins out of three thousand one hundred and forty-two. The two hundred and fifty-one excluded bins are the three held-out years, two thousand and nine, two thousand and seventeen and two thousand and twenty-three, together with the shadow of bins around them, plus the two ends of the time axis. The space rule says a cell counts if a current was ever observed there, which gives roughly eighty-six thousand seven hundred ocean cells. Multiplying the two gives a pool of about two hundred and fifty-one million possible anchors, and we draw from it uniformly with replacement, so there are no epochs and nothing is ever exhausted. A twenty-thousand-step run with two hundred and fifty-six examples per step touches five point one million anchors, which is two per cent of the pool. There is a subtlety worth stating out loud: although examples barely repeat, individual tensor values are re-read constantly, because each cell sits inside roughly seven hundred different cones. That factor of seven hundred is free today, but it is precisely the multiplier a materialized dataset would pay for in disk space.

The held-out years, and the shadow around them



Zoom — one year edge, in 5-day bins



Training — certified

The whole cone ($t-6 \dots t+2$) must avoid held-out bins. Enforced twice: by the admissibility rule, and by an independent brute-force certificate over 4,096 drawn anchors before the model is built — 0 violations, or the run refuses to start. Runs #536 and #537 both printed 0.

Evaluation — not certified

Only the anchor bin must be held out; the cone's context may reach into training bins. About 11 % of evaluation anchors sit within 6 bins of a year edge and straddle. Deliberate — "a held-out measurement, not a second training pool" — but an asymmetry.

The axis ends: cones that would run off 1982-01-01 or past 2024-12-31 are refused from every pool, never padded.

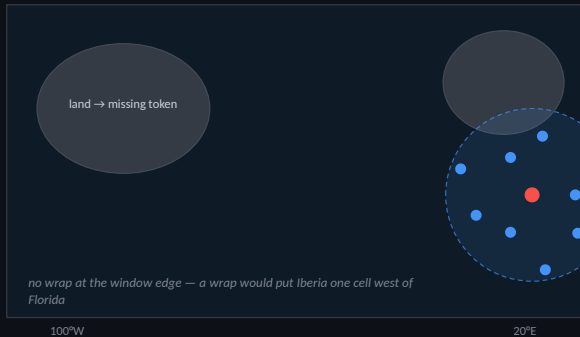
What is a training example?

6

Leakage, in one sentence: if the model could see a held-out bin while training, then the score we later measure on that held-out year would not be a test of anything. Three whole years are held out — two thousand and nine, two thousand and seventeen and two thousand and twenty-three — and they are shown in red on the timeline. Around each held-out year there is a shadow, drawn in amber: six bins before and two after. The shadow is that shape because a cone reaches six bins backwards and two bins forwards, so an anchor within six bins before a held-out year would pull held-out data into its history, and one within two bins after would pull it into its targets. For training this is enforced twice over: once by the rule that picks admissible anchors, and once again by an independent brute-force certificate that checks four thousand and ninety-six freshly drawn anchors before the model is even built, and refuses to start the run unless it finds zero violations. Runs number five hundred and thirty-six and five hundred and thirty-seven both printed zero. Evaluation is deliberately looser: only the anchor's own bin has to be held out, and its surrounding context is allowed to reach back into training bins, which happens for about eleven per cent of evaluation anchors. The reasoning is that an evaluation anchor is a held-out measurement rather than a second training pool, but it is an asymmetry, it is not certified, and slide thirteen proposes measuring the stricter variant once so we know what it costs.

The edge of the map, land, and the dateline

0-70°N



○ off-window dot — kept as a token, valid = false, value 0, and never a target

The global family (E-070) will wrap at the dateline — a 10-line change, planned.

What is a training example?

Three kinds of token

real
a measured value the model may read

masked
a real value we hid on purpose — and ask for back

missing
land, never observed, or off the window — never a target

Existence is enforced in exactly one place: one key-padding mask. A test flips an invalid dot's value and asserts the embedding is bit-identical.

The second boundary is the edge of the map. The model works inside a window that runs from one hundred degrees west to twenty degrees east and from the equator to seventy degrees north, and a cone near the edge inevitably asks for dots that fall outside it. Those dots are not dropped, and the number of tokens does not change: the dot is kept, its value is set to zero, and it is flagged as not valid. A single key-padding mask — one list saying which tokens attention may look at — then removes it, so existence is enforced in exactly one place rather than scattered through the code, and a test proves it by flipping an invalid dot's value and checking the embedding comes out bit-for-bit identical. An invalid dot is also never used as a target. Land cells and cells that were never observed become a missing token, which is deliberately a different thing from a mask token: missing means nobody ever measured this, while masked means we measured it and hid it on purpose to make the model guess. That distinction is information the model should have, so we give it three token kinds rather than two. One last detail: the window does not wrap around, because wrapping here would place Iberia one cell west of Florida; the planned global family, E-070, will wrap at the dateline, and that is about a ten-line change.

Observation gaps are represented, never imputed

$\approx$ **83 %**

of depth-channel dots carry the missing token — even inside the Argo era

Argo is live one bin per month — one pentad in six



five months of 5-day bins — filled bins are the ones Argo actually reports

22 of 43 years carry no live Argo bins at all.

- Missing is never forward-filled: telling the model the subsurface was observed on a day it was not would be a lie in the data.
- An unobserved target has weight exactly 0 — the decoder is never scored against a made-up zero.
- The token shape stays fixed (706 dots at every latitude); liveness rides in a flag, not in the shape.

Missingness is information — so it is a token, not a hole.

What is a training example?

8

The third boundary is about gaps in the observations themselves. Argo is the global fleet of drifting floats that measure temperature and salinity below the surface, and it reports roughly one five-day bin per month, so about eighty-three per cent of the depth-channel dots carry a missing token even during the years when Argo was fully operating. Before Argo, it is worse: twenty-two of the forty-three years in the record contain no live Argo bins at all. The rule we follow is that missing values are never forward-filled from the last observation. Filling them would tell the model that the subsurface was measured on a day when nobody measured it, and that is simply a lie written into the data. The same rule applies on the output side: if a target value was never observed, its weight in the loss is exactly zero, so the decoder is never rewarded or punished for matching an invented number. The shape of the example does not change with latitude or with era — there are always seven hundred and six dots — because a fixed shape keeps the batching simple; whether a value is live rides in a flag alongside it. The principle behind all of this is that missingness is itself information, and the model should be told about it rather than shielded from it.

What the example is actually used for

Hide

- **whole channels** — currents 50 %, others 30 %
- **a band of recent lags** — 30 % of examples, lags 1 ... l_0
- **a 90° sector** — 30 % of examples

Ask 382 questions

- 42** reconstruct the anchor's own values
- 84** predict the future values at $t+1$ and $t+2$
- 256** recover hidden dots from elsewhere in the cone

Score

Gaussian negative log-likelihood, weighted by family (A 1.0, B 2.0, C 1.5). The headline term reads the 32-number embedding z alone.

The snapshot twin ($L_{in} = 0$)

44 tokens, no history, still asked for $t+1$ and $t+2$ — the H1 control: does the embedding carry velocity?

The example is used for a self-supervised task, which means the data supplies its own answers and no human labels are needed. First we hide things: sometimes we hide whole channels, dropping the currents half the time and the other quantities about thirty per cent of the time; sometimes, in about three examples in ten, we hide a band of the most recent lags, from lag one out to some cut-off; and in another three in ten we hide a ninety-degree wedge of the surrounding disc, as if one direction had gone dark. Then we ask the decoder three hundred and eighty-two questions: reconstruct the forty-two values at the anchor, predict the eighty-four values one and two bins into the future, and recover two hundred and fifty-six of the hidden dots. The score is a Gaussian negative log-likelihood, which rewards the model for being right and for being honest about how sure it is, and the three channel families are weighted differently — one, two and one and a half. The headline term is computed from the thirty-two-number embedding alone, which forces that small vector to carry the useful information rather than letting the decoder cheat by peeking at the raw tokens. Notice that asking for t plus one and t plus two makes forecasting part of pre-training rather than something bolted on later. Finally there is a control we call the snapshot twin: the same model with the history length set to zero, so it sees forty-four tokens and no past at all, yet is still asked for the future. That control answers hypothesis one, which asks whether the embedding actually carries motion — velocity — or only a still picture.

The forecaster's example is a different object

A sequence of embeddings, not raw values

- Reads 144 lags (2 years) of embeddings at the anchor column, plus the outer spiral — 24 cells per lag from lag 7.
- Embeddings come from the materialized Z cache (17.4 GB), gathered on the fly per window.
- Off-window cells are zero-filled; windows must not touch any held-out bin, including each frame's teacher-forced target.
- That rule was derived once and then re-counted by brute force: 2,779 → 2,417 usable end-bins.

3,432

cells read per window — the new annulus stencil

20,880

cells read by the older full cylinder

Status: the annulus stencil is the one E-069 piece not yet written.

There is a second stage, and its training example is a completely different object from the one we have been describing. A stage-two example is not raw measurements at all: it is a sequence of embeddings, the small thirty-two-number summaries the codec produced. It reads one hundred and forty-four lags, which is two years of five-day bins, taking the embeddings at the anchor's own column plus an outer spiral of twenty-four cells per lag starting at lag seven. That comes to three thousand four hundred and thirty-two cells per window, against twenty thousand eight hundred and eighty for the older design that read a full cylinder — roughly six times cheaper. The embeddings themselves do come from a materialized cache, seventeen point four gigabytes of them, but the windows are still assembled on the fly. Its boundary rules are stricter in one respect: a window must not touch any held-out bin at all, including the target that each frame is teacher-forced on, meaning the true value fed back in during training. That rule was worked out on paper once and then re-counted by brute force, which cut the usable end-bins from two thousand seven hundred and seventy-nine down to two thousand four hundred and seventeen. It is worth saying plainly that these stage-two boundary rules were the source of the contamination lesson from August 2026, and that the annulus stencil is the one piece of E-069 still unwritten.

Generate on the fly, or materialize every example?

	Option 1 — generate on the fly (today)	Option 2 — materialize each example
Storage	0 per example	251 M × 19 KB ≈ 4.8 TB for the pool; one run's stream ≈ 97 GB
Duplication	none	700× — each cell sits inside ~700 cones
What an example is	a pure function of (tensor sha, geometry, anchor, mask seed)	bytes on disk, with boundary flags baked in
Boundary handling	the sampler + a certificate that runs every dispatch	the pipeline — but a sampler bug becomes permanent data
Changing the geometry	costs nothing	rebuilds everything
Inspectability	not inspectable without running code	examples are files you can open; sequential reads are fast
Runtime cost of option 1: 0.2–0.8 ms per anchor of host gather — about 1 hour of a 20,000-step run, not overlapped with the GPU today.		

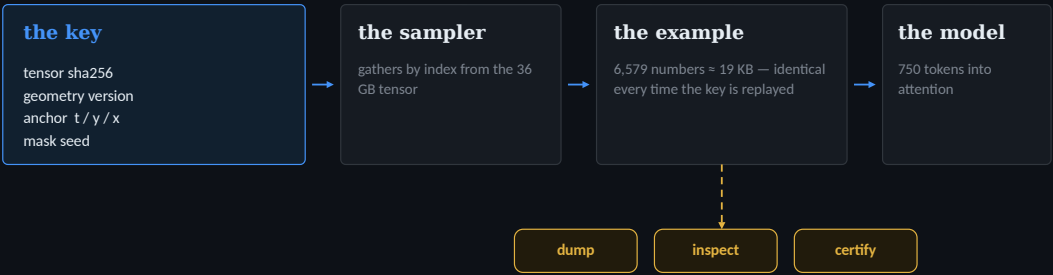
What is a training example?

11

Here are the two ends of the spectrum, side by side and as fairly as I can put them. Option one is what we do today: nothing is stored per example, so there is no duplication, and an example is a pure function of four things — the fingerprint of the tensor, the geometry version, the anchor’s position in time and space, and the seed used for masking. Boundary handling lives in the sampler and is checked by a certificate that runs before every dispatch, and changing the geometry costs nothing at all because nothing has to be rebuilt. Its real weakness is that you cannot inspect an example without running code, and there is a runtime cost: between zero point two and zero point eight milliseconds per anchor spent gathering on the host, which adds up to roughly an hour of a twenty-thousand-step run and is not currently overlapped with the graphics processor. Option two writes each example out, and the whole pool would be two hundred and fifty-one million examples of nineteen kilobytes each, which is about four point eight terabytes, or seven hundred times the size of the tensor it was derived from; even one run’s stream is about ninety-seven gigabytes. Any change to the geometry or the window means rebuilding all of it, and the boundary flags are baked into the bytes, so a bug in the sampler becomes permanent data rather than something you fix and rerun. To be fair to option two, its virtues are real: sequential reads are fast, boundary handling can live in the pipeline where it is easy to reason about, and examples are files you can simply open and look at. The cost is that your dataset becomes a derived cache of a thirty-six gigabyte source, seven hundred times larger than the source, that must be regenerated every time the recipe changes.

Materialize the key, not the bytes

An example is fully determined by a **small key**. Materializing the bytes is caching; materializing the key gives the same inspectability at zero storage — provided a tool turns a key back into the example, bit-identically.



The repo already works this way: the stage-2 cache is keyed by two hashes, and the JAX port is certified by gates rather than by re-reading weights.

The choice between generating and materializing is a false dichotomy, and this slide is the hinge of the deck. An example is completely determined by four small pieces of information: the sha256 fingerprint of the tensor, the version of the geometry, the anchor’s coordinates in time and space, and the seed used to decide what gets masked. Call those four things the key. Writing the nineteen kilobytes of an example to disk is therefore not creating data — it is caching a function of that key. So if we materialize the key instead, we get the same ability to point at, share and re-open a specific example, at zero storage cost. The one condition is that a tool must exist that turns a key back into exactly the same example, bit for bit, on demand — without that tool the key is just a promise. With it, we can dump an example, look at it, and certify it, which are the three things option two was really buying us. This is not a new discipline for this project: the stage-two embedding cache is already keyed by two hashes at once, and the port of the code to JAX, a numerical computing library, is trusted because it passes gates rather than because anyone re-read the weights by hand.

PROPOSAL

What I would do — about one day of work

1 A dump-by-key tool

Dump any example by key — or by "run #N, step s, batch index b", replaying the seeded stream — to NPZ, JSON and a picture. The Cones tab gains a "show a real example" mode.

2 A run manifest

Tensor sha, geometry json sha, seed and step count written into every run's record, so any example ever trained on is regenerable.

3 Materialize the evaluation set

The 1,024 evaluation anchors and their fixed mask draws, as a versioned file shipped with each run. This is where materialization pays.

4 An executable boundary spec

Property tests sweeping every edge row and column, both axis ends and every holdout edge; the certificate extended to evaluation anchors behind --eval-scope [anchor, window].

5 Materialize the observed mask

Once per tensor — a 2.2 GB bitmask — and precompute the row tables. Removes the double read and part of the 0.2-0.8 ms.

6 An escape hatch

--materialize-stream writes a run's exact examples (= 97 GB) for audits. Never for training.

Cost: about one day of implementation • no change to E-069's numbers • the evaluation file becomes an artefact.

What is a training example?

13

Here is the concrete proposal, six pieces, and none of them changes how training works. First, a tool that dumps any example given its key, or given a friendlier address like run number, step and batch index, by replaying the seeded stream; it writes an NPZ file, which is a compressed bundle of numeric arrays, a JSON file — JSON being a plain-text format for structured data — holding the metadata, and a picture, and the app's Cones tab gains a mode that shows a real example rather than a schematic. Second, a run manifest that records the tensor fingerprint, the geometry fingerprint, the seed and the step count, so that any example ever trained on can be regenerated years later. Third, materialize the evaluation set — the one thousand and twenty-four evaluation anchors and their fixed mask draws — as a versioned file shipped with each run; this is the one place where writing bytes genuinely pays, because it makes results comparable across different hardware backends and lets anyone inspect exactly what was scored. Fourth, an executable boundary specification: property tests that sweep every edge row and column, both ends of the time axis and every held-out year edge, plus the existing certificate extended to evaluation anchors behind a switch whose default is exactly today's behaviour. Fifth, materialize the observed mask once per tensor as a two point two gigabyte bitmask and precompute the row tables, which removes a double read and part of the per-anchor cost. Sixth, an escape hatch that can write out a run's exact examples, about ninety-seven gigabytes, for an audit — never for training. All together this is roughly one day of implementation, it changes none of E-069's numbers, and it turns the evaluation set into a real artefact.

Boundary handling under each option

BOUNDARY	ENFORCED TODAY	UNDER OPTION 2	HOW IT IS PROVEN
time / held-out years	the admissibility rule in the sampler	a filter at build time	certificate over 4,096 anchors, before every run
window edge	dot kept, valid = false; one key-padding mask	a flag baked into each stored example	test: flip an invalid dot, embedding is bit-identical
land / never observed	the missing token, at gather time	a missing flag written once at build	by construction, plus gather tests
dateline / wrap	no wrap; E-070 will wrap — a 10-line change	a full rebuild of every byte	geometry test (planned)
axis ends 1982 / 2024	cones refused from every pool, never padded	rows simply absent from the built set	admissibility count + the same certificate

Under option 1 the proof is a test that runs every dispatch. Under option 2 the proof is a build that ran once.

This matrix asks, for each boundary, where the rule is enforced today, where it would live if we materialized everything, and how we actually know it holds. The time boundary is enforced by the admissibility rule inside the sampler and proven by a certificate over four thousand and ninety-six anchors before every run; under option two it would become a filter that runs when the dataset is built. The window edge is handled by keeping the dot and marking it invalid, with a single key-padding mask doing the work, and it is proven by a test that flips an invalid dot and checks the embedding does not move. Land and never-observed cells become missing tokens at gather time, proven partly by construction and partly by gather tests. The dateline is simply not wrapped today, and the planned global family will wrap it. The ends of the time axis are handled by refusing those cones from every pool rather than padding them. The pattern down the right-hand column is the point of the slide: under option one, every one of those proofs is a test that runs on every dispatch, so a regression shows up the next time you press go. Under option two the proof is a build that ran once, months ago, and the repo's standing rule is to verify the artefact rather than the intention — a materialized dataset's flags are exactly an intention frozen at build time.

Four questions for you

Generation

Keep on-the-fly generation, as proposed — or materialize a run's stream by default?

Evaluation scope

Keep "anchor bin held out, context may straddle" — or measure the strict variant too?

A public gallery

A small gallery of real examples — say 50, about 1 MB — for the Cones tab and the paper. Yes or no?

Order of work

After run #537's H1 reading, or in parallel? It touches no training code path except the evaluation-set file.

Recommendation: parallel, small, and the strict evaluation variant measured once — so the straddle's effect is a number rather than an assumption.

Four decisions, and they are yours rather than mine. First, do we keep generating examples on the fly, as the proposal recommends, or do we start materializing each run's stream by default? Second, on evaluation scope: do we keep today's rule, where only the anchor's own bin must be held out and its context may straddle into training bins, or do we also measure the stricter variant in which the entire cone must be held out? Third, do we want a small public gallery of real examples — say fifty of them, about one megabyte in total — to sit on the Cones tab and to go into the paper? Fourth, when should this happen: after run five hundred and thirty-seven gives us its reading on hypothesis one, or in parallel with it? The work touches no training code path at all except the evaluation-set file, which is why parallel is realistic. My recommendation is to do it in parallel, keep it small, and measure the strict evaluation variant once, so that the effect of the straddle becomes a number we can quote rather than an assumption we keep repeating.